

The contribution is the combination and the documented trade-offs, not a claim that every individual feature is novel.

**The main functional areas are:**

**membership and identity** (registration, approval, digital ID, certificates, profiles, privacy controls, searchable directory);

**accounting and payments** (fees, dues, donations, configurable payment instructions, manual payment proof, administrative verification, receipts and audit history);

**events and participation** (publication, registration, capacity control, waitlisting, QR attendance, budgets and moderated photo submissions);

**communication and networking** (news, notices, targeted communication, real-time messaging, notifications, jobs and mentorship); and

**governance and administration** (constitution versioning, Executive Committee records, amendment voting, polls, voter-roll preparation and election-document support).

The platform supports governance procedures but does not replace the Election Commission or claims to provide a legally sufficient **electronic election system**.

**Portability, legacy data and configuration**

The persistence boundary is provider-neutral to the extent required by the specification: PostgreSQL is the deployment provider, and SQLite is used for local development and tests without changing business source code. The deployment is also container-portable and is intended to run on a container host without a proprietary platform dependency.

Legacy handling is part of the design rather than an afterthought. Migration bootstrap detects databases created by the earlier schema mechanism, baselines existing objects when appropriate, apply only genuinely pending migrations, and logs failures. Membership and financial records use archival deletion and query filters, so ordinary screens hide archived rows without destroying audit evidence. Full export in an open format is a portability requirement.

The configuration model has two planes:

- **Runtime configuration:** organization name, branding, contact details, currency, feature flags, workflow settings, membership presentation, payment instructions and public content can be changed by authorized administrators without redeployment.
- **Boot profile configuration:** a profile pack supplies fresh-deployment defaults, seed data, assets, documents, email templates, lookups, membership tiers and SEO information through `ORG_PROFILE`.

The profile-pack goal is deployment of a new organization from configuration and assets rather than a source fork. The current prototype already provides substantial runtime and profile support, while the remaining brand-literal cleanup and full zero-source-change onboarding remain explicit implementation work. This distinction prevents a planned white-label capability from being presented as fully proven.

**Reusable platform foundation**

The platform is **built as a reusable foundation** rather than a one-off application. Authentication, role-based authorization, accounting and ledger services, audit logging, file handling, SignalR notifications, organization configuration, shared validation, reusable Angular controls (Shared frontend controls include page headers, search fields, filters, data tables, pagination, loading states, empty states, confirmation dialogs and shared form patterns) and Flutter widgets, and CI/CD are designed as organization-agnostic services, separate from Association-specific rules and configuration. Runtime settings such as branding, currency, feature flags and payment instructions can be changed by administrators without redeployment, and a boot profile pack can seed a fresh deployment's defaults, documents and content.

**Non-functional requirements** cover performance, security, reliability, usability, maintainability, portability, recovery, auditability and privacy, evaluated through quality-attribute scenarios such as directory latency, session

invalidation on status change, dependency-failure isolation, architecture-rule enforcement and recovery time. The prototype already includes the backend, web app, mobile app, authentication, accounting, membership, events, governance and CI/CD. Remaining work covers traceability closure, measured coverage, security and usability evidence, performance evaluation and documentation.

### **Development approach**

The project uses Design Science Research with incremental software development:

1. specify the requirement and acceptance condition;
2. model the workflow, state or business rule;
3. implement it across the required layers;
4. test the implementation;
5. trace the requirement to its evidence;
6. evaluate the result and record limitations.

Specification-driven development is supported by a requirements traceability matrix, use cases, quality-attribute scenarios, domain constraints, acceptance criteria and tests. The architecture is evaluated against measurable scenarios rather than selected only for implementation convenience.

The design uses Clean Architecture, dependency inversion, dependency injection, explicit application contracts and DTOs, typed API communication, role-based access control, boundary validation, global archive/query-filter behavior, centralized design tokens, reusable controls, configuration-driven organization profiles and shared services instead of duplicated feature logic.

### **Testing, coverage and evaluation**

The project uses backend unit tests for services, validators and business rules; backend integration tests for database and API behavior; EF Core SQLite and in-memory test configurations; Angular unit tests using Vitest; Flutter widget and service tests; Playwright end-to-end tests; authentication, authorization and privacy tests; API contract and compatibility checks; selected visual regression tests; and static analysis, type checking and build verification. Evidence includes backend statement and branch coverage through Coverlet, requirement coverage through the traceability matrix, critical business-rule tests, API and end-to-end results, security assessment, usability findings, performance measurements, maintainability evidence and deployment-cost assessment.

The quality scenarios include directory latency, cold mobile page load, session invalidation, authorization isolation, dependency-failure containment, maintainer change effort, architecture-rule enforcement, recovery time, accessibility and vote replay rejection. These scenarios make proposals about verifiable system qualities, not only stored records and screens. Results will be reported by suite and requirement category. No unsupported combined coverage percentage will be claimed. The intended target is that all Must requirements are traced to passing tests; requirements without sufficient evidence will be reported as uncovered or limited.

### **Requirements engineering and specification-driven development**

Requirements are handled as an engineering activity with its own outputs, not as notes kept before coding. They are elicited from four sources: stakeholder interviews, the Association's current manual process, its constitution and election documents, and software-engineering standards. They are specified to ISO/IEC/IEEE 29148 in the "shall" form, each with an identifier, source and priority. Quality requirements are classified by ISO/IEC 25010 and restated as testable quality-attribute scenarios. Constitutional rules become domain constraints that the software must not break, and stakeholders cannot negotiate them away.

Specification-driven development is carried out with GitHub Spec Kit. A short specification constitution sets five rules every spec must follow: keep the existing product contract, trace shared behaviour across the API, web and mobile clients, keep layer boundaries clear, treat security and privacy as requirements, and back every claim with a repository check. The work proceeds in Spec Kit's order: specify, clarify, plan, break into tasks, implement. Each domain spec holds user stories with acceptance scenarios, functional requirements, success criteria, an evidence table linking each requirement to its controller, service, screens and test, and a gaps list for requirements that have no test yet.

The evaluation reports how far the specs and the code agree. That covers requirements with passing tests, requirements with only a recorded gap, and open clarification points. The specs for the existing modules were